CORTE 1 - TALLER 1
Inteligencia Estratégica con Árboles de Juego

Estructura de datos no lineales
Por : Guillermo Andres De Mendoza Corrales

Fecha de entrega maxima: Lunes, 23 de febrero
Lugar de entrega: repositorio GIT de cada grupo

Tema: Árboles n-arios y Algoritmos de Búsqueda

1. Introducción

En este taller, diseñaremos un programa capaz de "prever el futuro" en una partida de Triki. Utilizaremos una estructura de **Árbol de Estados**, donde la raíz representa el tablero actual y cada hijo representa una jugada posible.

2. El Desafío

Deben desarrollar una aplicación que reciba como entrada un tablero de Triki en un estado cualquiera. A partir de ahí, el programa debe generar todas las secuencias de jugadas posibles hasta que el juego termine.

Regla de Oro (Poda): Si en un nodo (tablero) se detecta que hay un ganador (X o O) o un empate (tablero lleno), ese nodo se convierte en una **hoja**. No se deben generar más hijos a partir de un estado finalizado.

3. Requisitos Técnicos

A. Representación del Tablero

El tablero debe ser representado de forma eficiente (por ejemplo, una matriz de 3x3 o un arreglo de 9 posiciones) (recomiendo numpy).

B. Estructura del Nodo

Cada nodo del árbol debe contener:

1. El estado del tablero en ese momento.
2. El jugador al que le toca mover.
3. Referencia al nodo padre (para ver el recorrido)
4. Una lista de referencias a sus nodos hijos (jugadas posibles).

C. Lógica de Implementación

El algoritmo debe ser recursivo y seguir estos pasos:

- **Paso 1:** Verificar si el tablero actual tiene un ganador o está lleno. Si es así, marcar como "Hoja" y terminar esa rama.
- **Paso 2:** Si el juego continúa, identificar todas las casillas vacías.
- **Paso 3:** Para cada casilla vacía, crear un nuevo nodo hijo con el movimiento del jugador en turno.
- **Paso 4:** Repetir el proceso para cada hijo (llamada recursiva).

4. Entregables

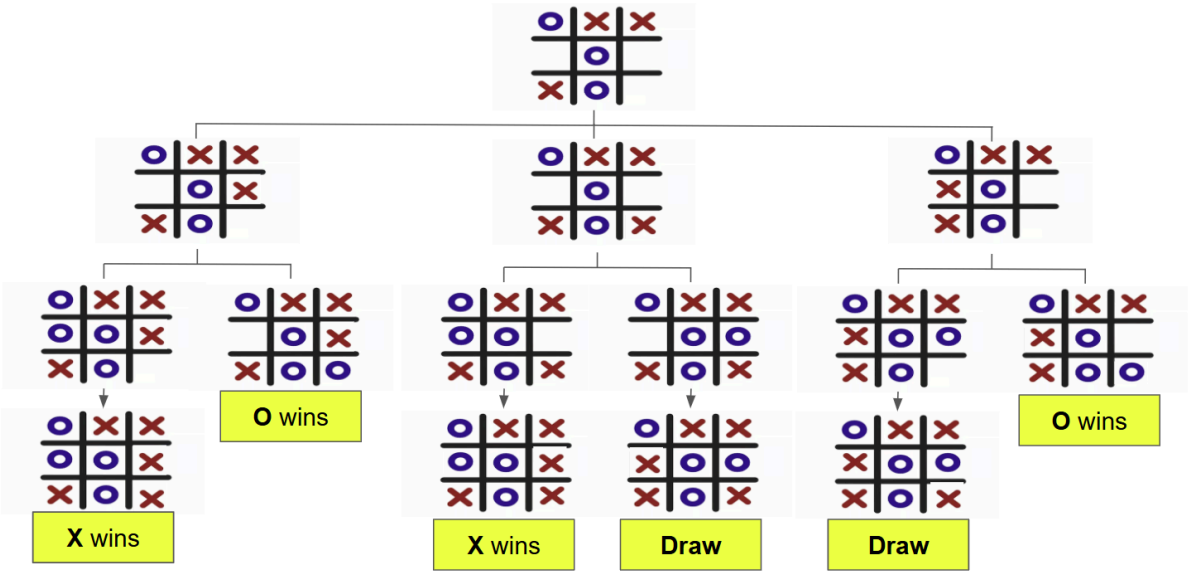
1. **Código Fuente:** Implementación de la estructura del árbol y el algoritmo de generación.
2. **Visualización:** Una función que permita imprimir el árbol (o al menos una rama completa) en la consola para verificar un resultado.

5. Salidas del programa

El programa al finalizar debe:

1. Indicar cual es la profundidad minima y maxima
2. Cuantas hojas terminan en victoria para X, O y empates
3. Se debe poder visualizar el estado final de un juego (verificable)
4. Se debe poder visualizar el recorrido de un juego (Pasos) (verificable)

Ejemplo:



Profundidad minima: 2

Profundidad maxima: 3

Victorias X: 2

Victorias O: 2

Empates: 2